hello

VSE TA assignment problem

econ622 project idea | Kitty Wu

it's me, Claude and a dream...

motivation

For each academic session, the VSE currently allocates TAs using a deferred acceptance matching procedure — I heard they are revamping the system soon.

motivation

For each academic session, the VSE currently allocates TAs using a deferred acceptance matching procedure — I heard they are revamping the system soon.

Think of this as an exploration and mechanical exercise for tailored matching problems in practice

- TA matching as a discrete constrained optimization problem, offer some thoughts and comparisons to alternative methods…
- Something a little less traditionally ‘academic’…

Current algorithm implemented by James Yu @jbrightuniverse on github!

matching problem

$\mathrm{I} = \text{students}$

$\mathrm{J} = \text{courses}$

$u_{\mathrm{ij}} = \text{student i utility from course j}$

$\mathrm{v_{ji}} = \text{course j utility from student i}$

Rankings: $\mathrm{P_i} = \text{student i ranking over courses}$; $\mathrm{P_j} = \text{course j ranking over students}$

Decision variables: $x_{ij} \in \{0,1\}$ where $x_{ij} = 1 \; if \; a \; student \; is \; assigned \; to \; course \; j$

Constraints

- Each student is assigned at most once
- Some courses will always reject a student ("blacklist")
- PhD requirement: $\mathrm{d_j} = 1 \rightarrow \text{student i assigned only if } \mathrm{phd_i} = 1$
- Capacity: $\mathrm{c}_j = \text{capacity per course}$

project roadmap

Goal: using the VSE TA allocation setting to build a simple proof-of-concept framework that implements the current approach alongside alternative TA matching algorithms / computational methods, simulate preference data, and compare

01 — replicate the current VSE deferred acceptance mechanism

02 — adapt serial dictatorship to this setting using existing packages

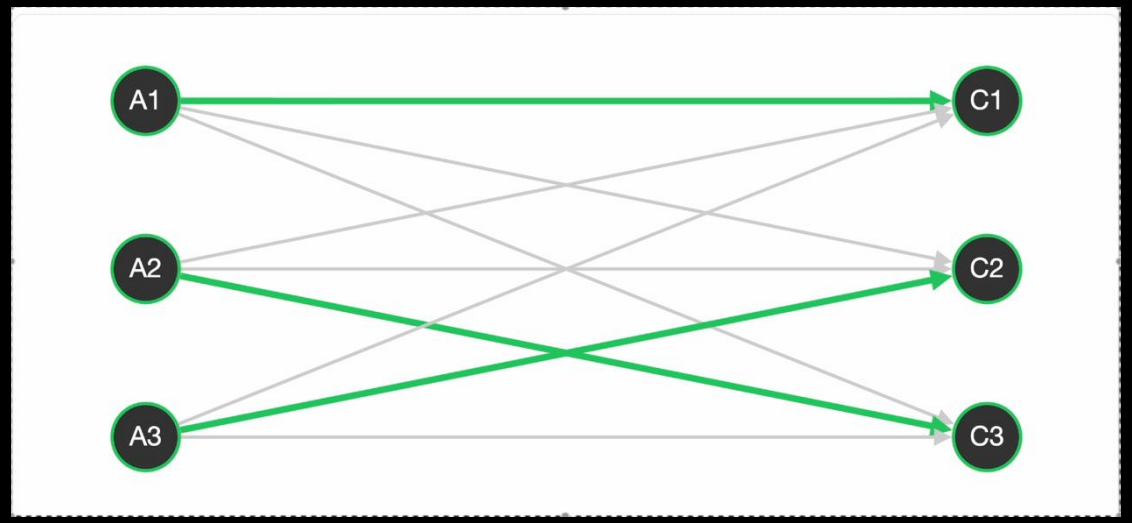
03 — formulate as a MIP: how would a social planner allocate TAs?

04 — simulate preference data using a latent skill vector DGP

05 — evaluate: stability, fairness, strategy-proofness, runtime, etc...

01. deferred acceptance (Gale – Shapley)

Students propose to their top-ranked course → courses tentatively accept or reject based on their own rankings → repeat until stable.
At the VSE: many-to-one converted to one-to-one via allocation blocks

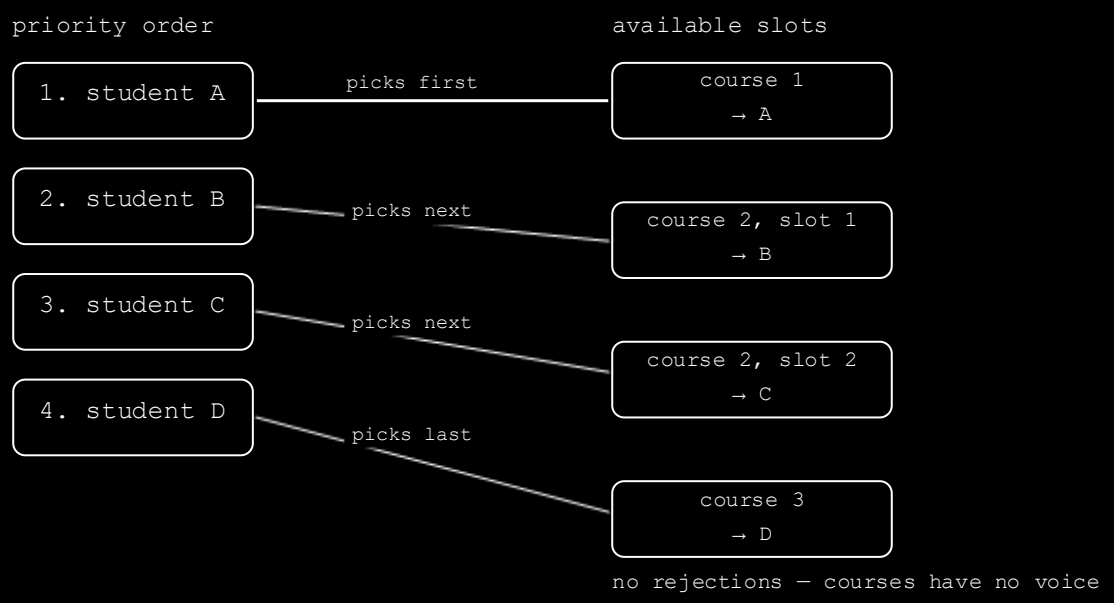


Key properties

- Student ('proposer') – optimal
- Strategy-proof for students
- Stable / no blocking pairs

02. serial dictatorship

Students pick in order of priority. First student gets their top choice, second student gets their top choice from what remains, and so on until all slots are filled.



Key properties

- Strategy-proof — your best move is always to pick your true top choice
- Pareto efficient — no student can be made better off without making another worse off
- BUT not stable — can produce blocking pairs

03. mixed integer programming (MIP)

Reframe allocation as a constrained optimization problem — how would a ‘social planner’ assign TAs, **assuming preferences are reported truthfully.**

MIP problem: consists of a linear objective function and linear constraints, similar to linear programming, but with the added integer restrictions

Applications: extensively used for complex scheduling, planning, logistics, and resource allocation problems

Objective functions

- *student utilitarian* — maximise sum of student utilities
- *bilateral utilitarian* — weighted trade-off between student and course welfare
- *egalitarian* — maximise the worst-off student's rank (min-max)

03. MIP implementation

The optimization is implemented in Python using CVXPY as the modelling interface and Gurobi as the solver backend.

MIP (exact)

- $x_{ij} \in \{0,1\}$ — solved exactly via Gurobi
- tractable at small sample scale: e.g. 20 students × 8 courses = 160 binary variables

LP relaxation (approximate)

- relax $x_{ij} \in [0,1]$ — now a continuous linear program
- solve in polynomial time, then round fractional assignments back to binary
- faster, and opens the door to gradient-based methods

04. data generating process

Key properties

$\varphi_{\mathrm{j}} \in \mathbb{R}^{\mathrm{k}}$ *course j skill / trait requirements*
$\theta_{\mathrm{i}} \in \mathbb{R}^{\mathrm{k}}$ *student i skill* / trait *vector*

... each trait could reflect quant skill, coding skill, etc..

Utility randomly drawn

$$u_{\mathrm{ij}} = \theta_i \cdot \varphi_{\mathrm{j}} + \varepsilon_{\mathrm{ij}}, \qquad \varepsilon_{\mathrm{ij}} \sim N(0, \sigma^2)$$
$$v_{\mathrm{ji}} = \theta_{\mathrm{i}} \cdot \varphi_{\mathrm{j}} + \eta_{\mathrm{ji}}, \qquad \eta_{\mathrm{ji}} \sim N(0, \sigma^2)$$

... preferences obtained from ranking utility scores

Unmatched students are a possible outcome (-1)
Agents named: Greek letter students (Alpha, Beta, ...) × ECON course codes (ECON101, ...)

Current / default assumptions:

- **20 students** (40% PhD → ~8 PhD students)
- **8 courses** (25% require PhD TA → ~2 courses)
- **k = 3** latent skill dimensions
- **σ = 0.5** noise level
- **capacities** drawn uniformly from (1, 3) per course
- **rejection fraction** of 0.1

next steps

1. Tweak existing algorithms with additional constraints / assumptions
    1. Sparse preferences?
    2. How to break ties?

2. Implement optimization / MIP
    1. LP relaxation / differentiation extension likely a stretch goal

3. Evaluate and compare
    1. Devise unit tests to evaluate key criteria where possible (e.g. check for no blocking pairs)
    2. Consider — Stability? Fairness? Strategy – proofness? How often unmatched?

?

github @kittywu123
Instagram @kittyc.att
linkedin.com/in/kitty-jt-wu/